

# Séries temporais e sistemas de recomendação

Fabio Santiago
fabio.santiago@facens.br




# Sistemas de recomendação baseados em modelos

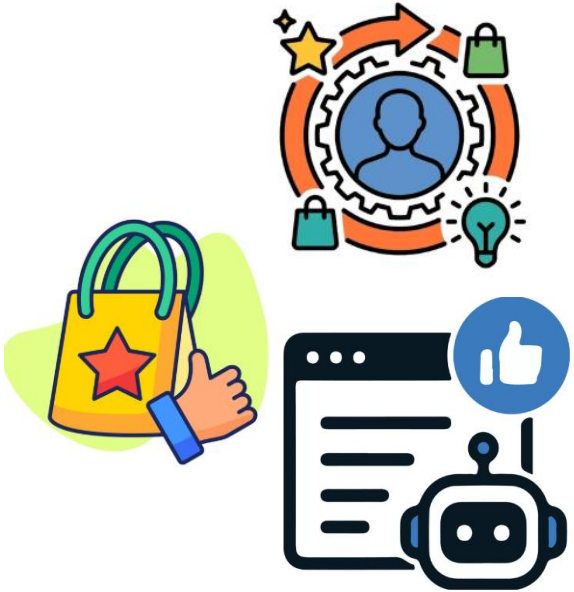
# Cronograma

- Contextualização: por que recomendar?
- Da filtragem baseada em memória à baseada em modelo
- O problema central: fatoração de matrizes
- Algoritmo SVD

# Por que recomendar?

**O cenário real**

Imagine que você entra na **Netflix**, na **Spotify** ou na **Amazon**. Há milhões de itens disponíveis, mas você só vê algumas dezenas. **Quem escolhe o que você vê?** Um sistema de recomendação.



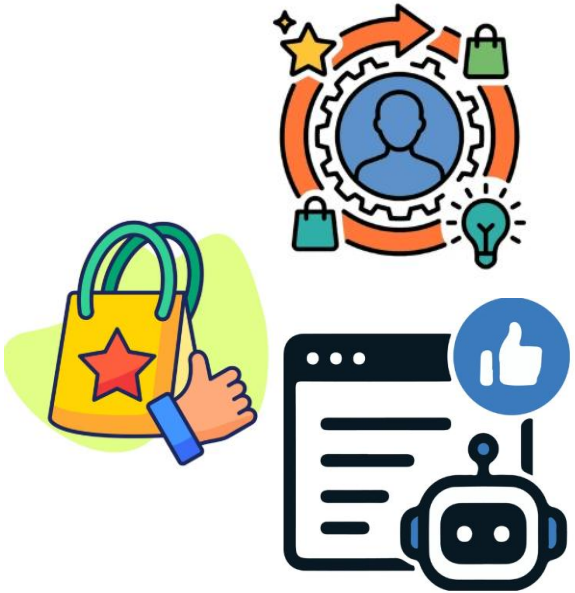
A Netflix estima que **80% do conteúdo assistido** vem de recomendações. A Amazon atribui **35% das vendas** ao seu motor de recomendações. Não é um detalhe — é o coração do produto.

# O problema fundamental

Dado um conjunto de usuários $\mathcal{U}$ e itens $\mathcal{I}$, queremos prever a **preferência** de um usuário $u$ por um item $i$ que ele **ainda não consumiu**.

Formalmente, temos uma **matriz de interações**

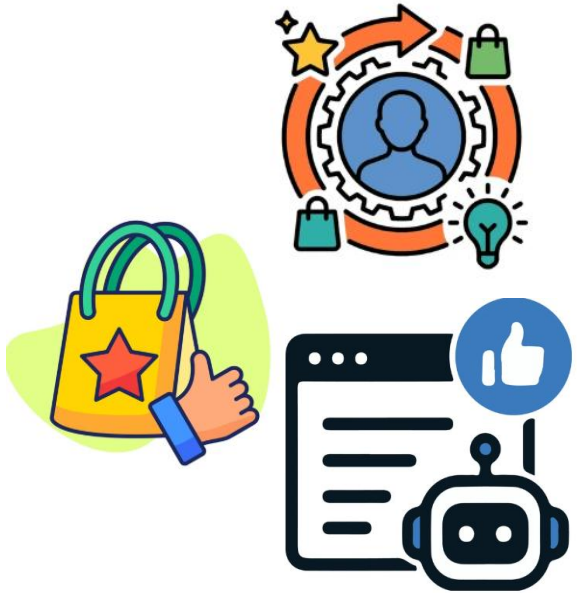
$$R \in \mathbb{R}^{|\mathcal{U}| \times |\mathcal{I}|}$$



# O problema fundamental

Os ? são **missing values** — o que queremos prever.

| | Star Wars | Matrix | Titanic | Interestelar |
|---|---|---|---|---|
| Ana | 5 | 4 | ? | 5 |
| Bruno | 4 | ? | 2 | 5 |
| Carla | ? | 5 | 1 | ? |
| Diego | 5 | 4 | ? | ? |



# O desafio da esparsidade

**Problema central**: Em sistemas reais, essa matriz é **extremamente esparsa**. A Netflix tem ~99% de valores ausentes. A Amazon, mais de 99,9%.

Isso significa que **não podemos simplesmente preencher com médias** — perderíamos toda a riqueza das preferências individuais. Precisamos de algo mais sofisticado.



# Da Filtragem Baseada em Memória à Baseada em Modelo

**Filtragem baseada em memória**

**Ideia**: "Se Ana e Bruno gostaram dos mesmos filmes, e Bruno gostou de Interestelar, talvez Ana também goste."

Duas variantes:

- **User-based**: encontra usuários similares a $u$e recomenda o que eles gostaram.
- **Item-based**: encontra itens similares a $i$e recomenda para quem gostou de $i$.

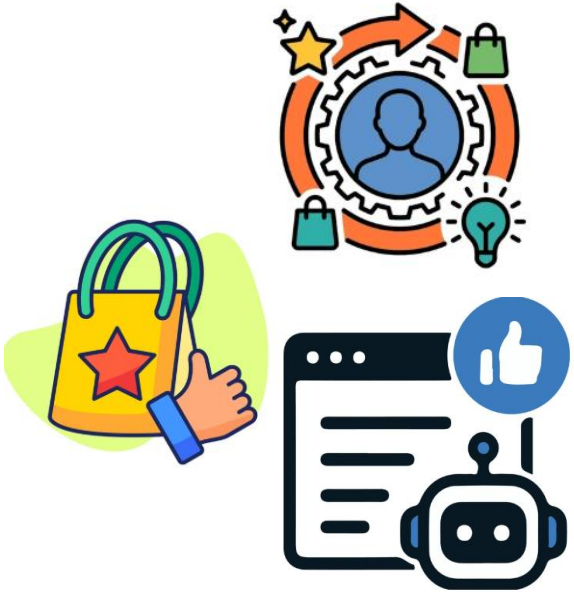


# Da Filtragem Baseada em Memória à Baseada em Modelo

**Filtragem baseada em memória**

**Limitações**:

- Não escala bem (precisa calcular similaridades em tempo real).
- Sensível a esparsidade.
- Não generaliza bem para novos usuários/itens (cold start).



# Da Filtragem Baseada em Memória à Baseada em Modelo

**Filtragem baseada em modelo**

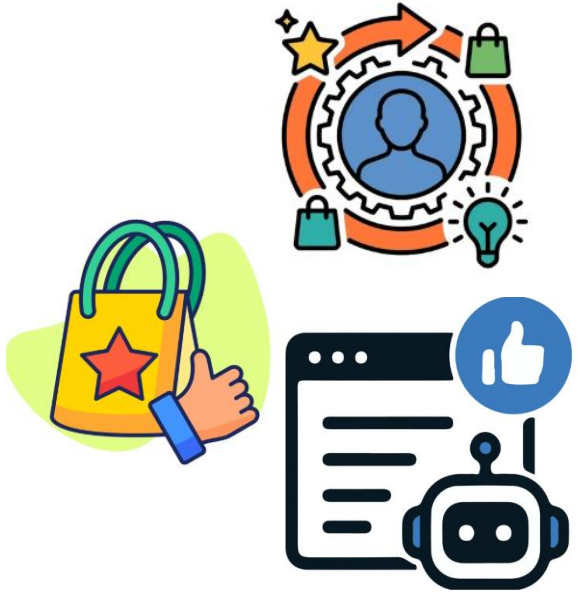
**Ideia**: Em vez de olhar para vizinhos, **aprenda um modelo** que explique as preferências.

Essa é a essência da **fatoração de matrizes**: projetar usuários e itens em um **espaço latente** de dimensão $k$(tipicamente 10 a 200).



# Da Filtragem Baseada em Memória à Baseada em Modelo

**Formalização da filtragem baseada em modelo**



Aprender vetores latentes $\mathbf{p}_u \in \mathbb{R}^k$(usuário) e $\mathbf{q}_i \in \mathbb{R}^k$(item) tais que:

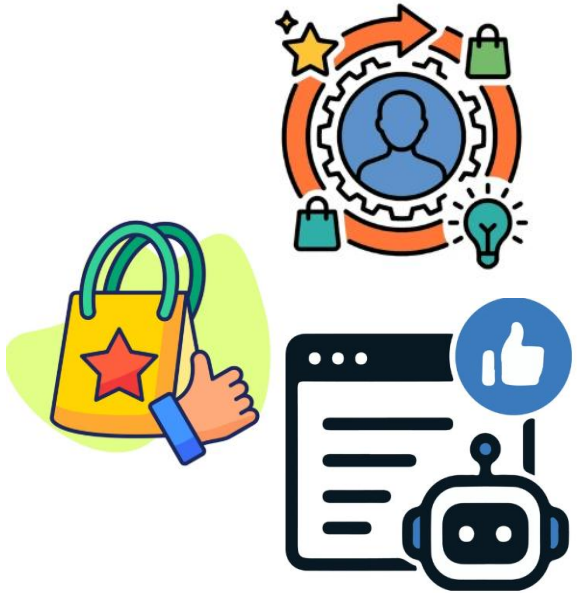
$$\hat{r}_{ui} \approx \mathbf{p}_u^T \mathbf{q}_i$$

Com viés (mais realista):

$$\hat{r}_{ui} \approx \mu + b_u + b_i + \mathbf{p}_u^T \mathbf{q}_i$$

# Da Filtragem Baseada em Memória à Baseada em Modelo

**Formalização da filtragem baseada em modelo**



sendo:

- $\mu$: média global dos ratings
- $b_u$: viés do usuário (Ana é generosa? dá notas altas?)
- $b_i$: viés do item (Interestelar é universalmente aclamado?)

# Da Filtragem Baseada em Memória à Baseada em Modelo

**Função objetivo**

Minimizar o erro quadrático com regularização:

$$\min_{\mathbf{P},\mathbf{Q}} \sum_{(u,i)\in\mathcal{K}} \left(r_{ui} - \hat{r}_{ui}\right)^2 + \lambda \left(\|\mathbf{p}_u\|^2 + \|\mathbf{q}_i\|^2 + b_u^2 + b_i^2\right)$$



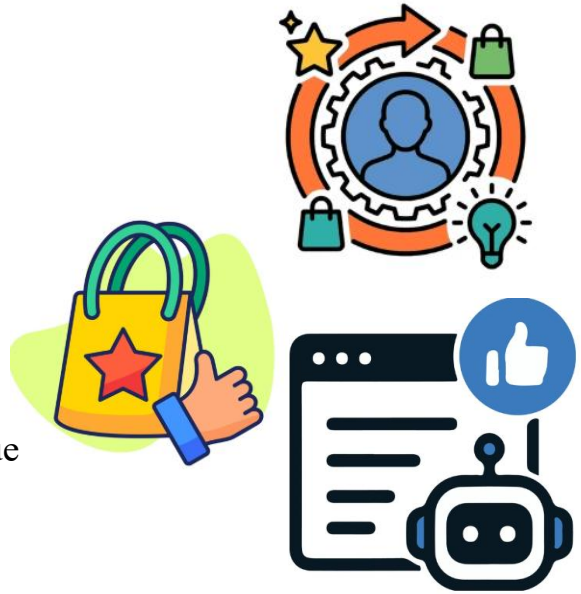
O desafio: essa função é **não convexa** em **P** e **Q** simultaneamente. É aqui que entram **ALS, SVD e SGD**.

# Da Filtragem Baseada em Memória à Baseada em Modelo

**Função objetivo**

Minimizar o erro quadrático com regularização:

$$\min_{\mathbf{P},\mathbf{Q}} \sum_{(u,i)\in\mathcal{K}} (r_{ui} - \hat{r}_{ui})^2 + \lambda \left( \|\mathbf{p}_u\|^2 + \|\mathbf{q}_i\|^2 + b_u^2 + b_i^2 \right)$$



O desafio: essa função é **não convexa** em **P**e **Q**simultaneamente. É aqui que entram **ALS, SVD e SGD**.

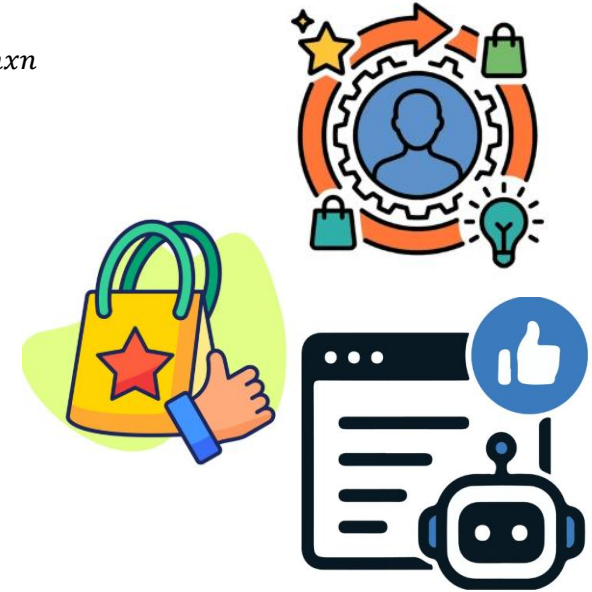
# Algoritmo SVD

A Singular Value Decomposition (SVD) clássica, decompõe uma matriz R ∈ $\mathbb{R}^{mxn}$

$$R = U\Sigma V^T$$

sendo:

- $U \in \mathbb{R}^{m \times m}$: vetores singulares à esquerda (usuários).
- $\Sigma \in \mathbb{R}^{m \times n}$: valores singulares (importância de cada fator).
- $V \in \mathbb{R}^{n \times n}$: vetores singulares à direita (itens)



# Algoritmo SVD

**O problema: missing values**

**Ponto crítico**: A SVD clássica **exige uma matriz completa**. Mas nossa matriz *R* tem 99% de valores ausentes!



# Algoritmo SVD

**Soluções**:

**a) Imputação**: preencher os ausentes com a média (ou zero) e aplicar SVD.

- Simples, mas **enviesa** os resultados e distorce a estrutura.

**b) FunkSVD**: aprender os fatores diretamente, ignorando os ausentes.

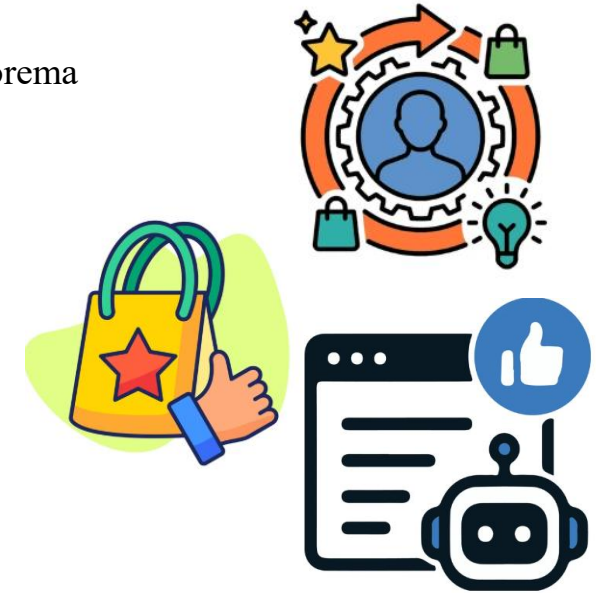
- Muito melhor. É a base das técnicas modernas



# Algoritmo SVD

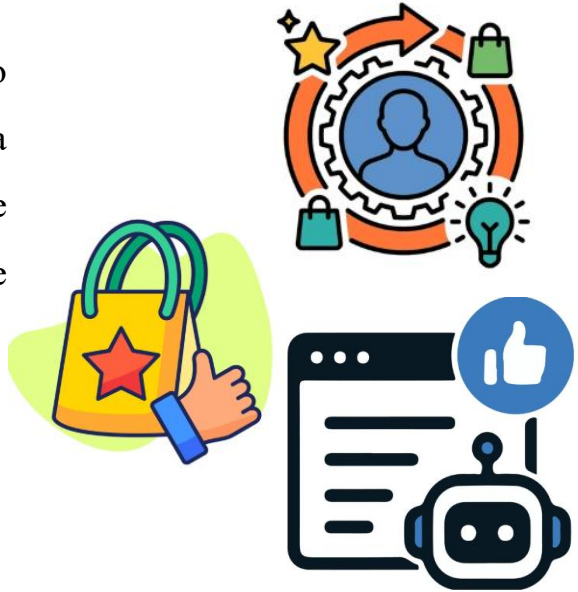
**Truncando** em $k$ dimensões, obtemos a **melhor aproximação de posto** $k$(Teorema de Eckart–Young):

$$R = U_{\mathrm{k}}\Sigma_{\mathrm{k}}V_k^T$$



# Algoritmo SVD- Teorema Eckart-Young

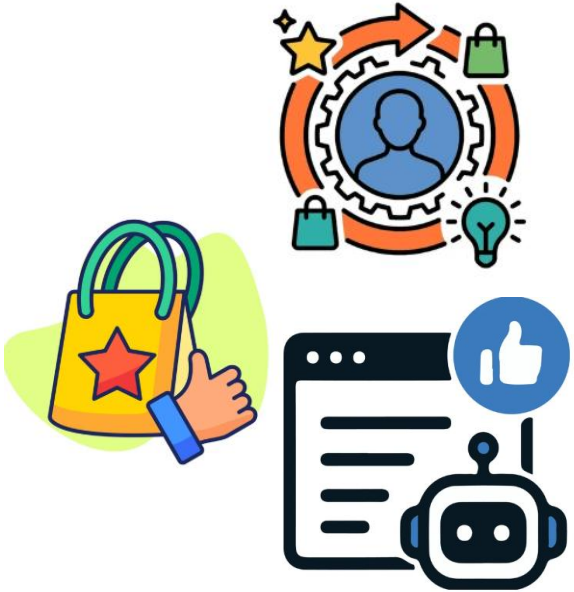
O teorema de Eckart–Young é um resultado fundamental da álgebra linear que, no contexto de sistemas de recomendação, fornece a base matemática para a técnica de **fatoração de matrizes via SVD (Decomposição em Valores Singulares)**. Ele garante que, ao aproximar a matriz de interações usuário-item por uma matriz de posto (rank) menor, o erro dessa aproximação é minimizado.



# Algoritmo SVD- Teorema Eckart-Young

**O que o Teorema diz**

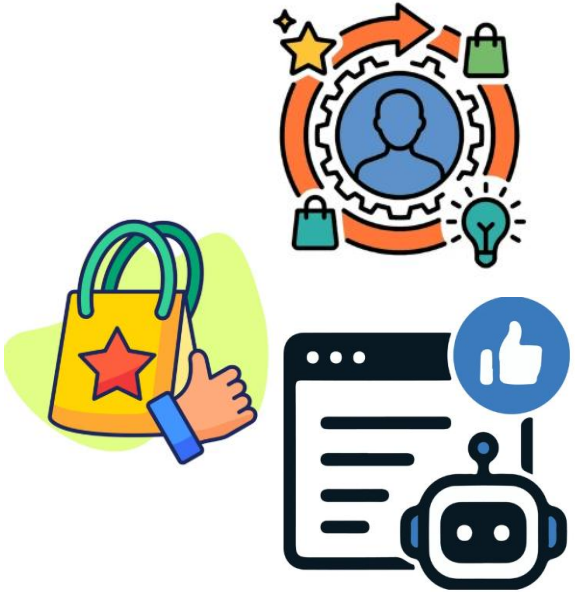
Formalmente, o teorema estabelece que a melhor aproximação de posto $\boldsymbol{k}$ para uma matriz $\boldsymbol{A}$ (no sentido da norma de Frobenius ou espectral) é obtida truncando-se a sua SVD para manter apenas os $\boldsymbol{k}$ maiores valores singulares . Em outras palavras, se você decompõe a matriz $\boldsymbol{A}$ em $\boldsymbol{A = U\Sigma V^T}$, a matriz $\boldsymbol{A_k = U_k \Sigma_k V_k^T}$ é aquela que minimiza o erro $\| \boldsymbol{A - A_k} \|$ entre todas as matrizes de **posto** $\boldsymbol{k}$.



# Algoritmo SVD- Teorema Eckart-Young

**Por que isso importa em Recomendação**

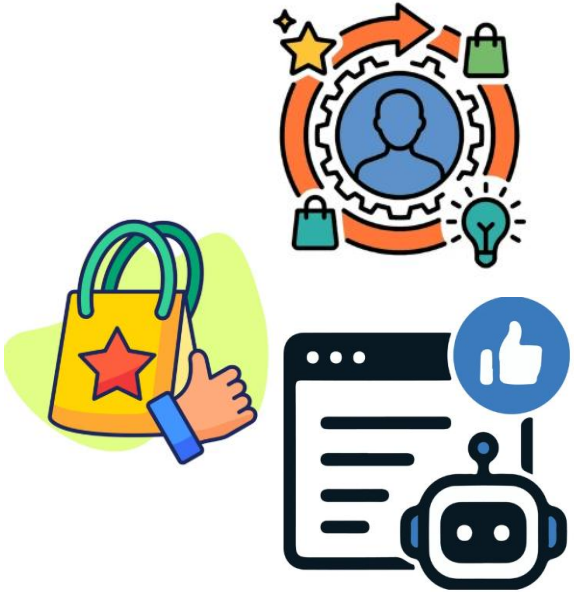
Em sistemas de recomendação, a **matriz de avaliações (ou interações)** *R*é tipicamente muito esparsa (a maioria dos usuários avaliou poucos itens) e, teoricamente, de posto alto. A hipótese central é que o **gosto dos usuários não é aleatório**, mas determinado por um número pequeno de **fatores latentes** (por exemplo, gênero de filme, época, ator preferido)



# Algoritmo SVD- Teorema Eckart-Young

**Por que isso importa em Recomendação**

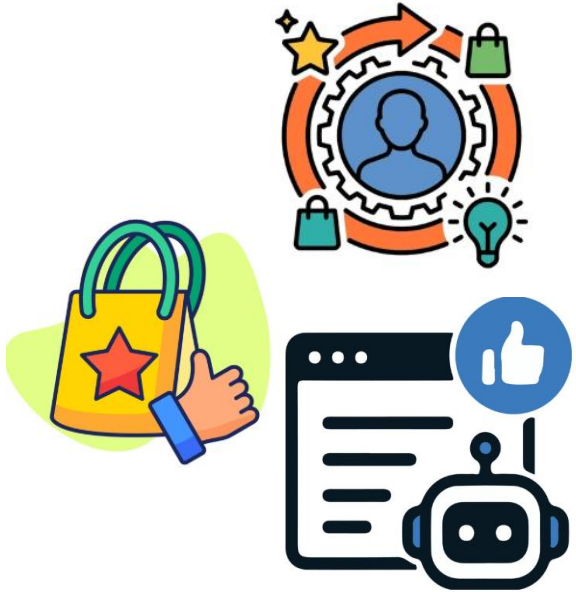
Portanto, em vez de tentar prever cada nota de forma independente, busca-se uma **aproximação de posto baixo** $R_k$ que capture a estrutura subjacente. O teorema de Eckart–Young justifica essa abordagem: ao usar a SVD truncada, estamos **garantidamente** escolhendo a matriz de posto $k$ que melhor se ajusta aos dados observados, minimizando o erro quadrático .



# Algoritmo SVD- Teorema Eckart-Young

**Por que isso importa em Recomendação**

Na prática, os valores singulares menores (que são descartados) geralmente representam ruído ou variações idiossincráticas, enquanto os maiores capturam os padrões de preferência compartilhado.



# Algoritmo SVD

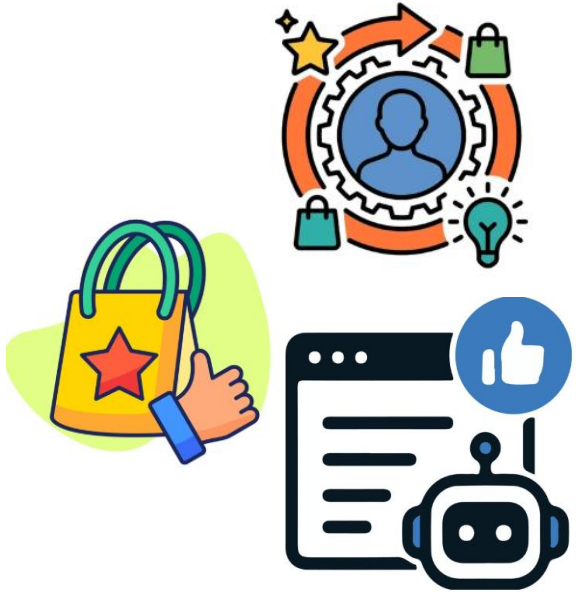
| Vantagens | Desvantagens |
| --- | --- |
| Base teórica sólida | SVD pura não lida com missing |
| FunkSVD/SVD++ muito precisos | Dependem de SGD (tuning) |
| Capturam fatores interpretáveis | Custo de memória |
| Referência em benchmarks | Menos escalável que ALS |



# Algoritmo SVD

**Quando usar SVD?**

- **Feedback explícito** (ratings).
- Necessidade de **alta precisão**.
- Datasets densos o suficiente.
- Não é ideal para feedback implícito massivo (prefira ALS).



# Sistemas de recomendação

Implementando .....



# Sistemas de recomendação


**BIBLIOGRAFIA BÁSICA**

HYNDMAN, R. J.; ATHANASOPOULOS, G. Forecasting: Principles and Practice. 3. ed. Melbourne: OTexts, 2021.

BOX, G. E. P.; JENKINS, G. M.; REINSEL, G. C.; LJUNG, G. M. Time Series Analysis: Forecasting and Control. 5. ed. Hoboken: Wiley, 2015.

AGGARWAL, C. C. Recommender Systems: The Textbook. Cham: Springer, 2016. 6.

**BIBLIOGRAFIA COMPLEMENTAR**

RICCI, F.; ROKACH, L.; SHAPIRA, B. Recommender Systems Handbook. 2. ed. Cham: Springer, 2022. • GRUS, J. Data Science do Zero. 2. ed. Rio de Janeiro: Alta Books, 2020.

ALMEIDA, J. M. et al. Sistemas de Recomendação. Rio de Janeiro: LTC, 2016. • MORETTIN, P. A.; TOLOI, C. M. C. Análise de Séries Temporais. 3. ed. São Paulo: Blucher, 2018.

BROWNLEE, J. Deep Learning for Time Series Forecasting. Melbourne: Machine Learning Mastery, 2018.


Facens
AQUI TEM ENGENHARIA